

Rocq projects: further topics

Formal Software Verification

Aarhus University

2026

These projects are experimental. No student has worked through one yet, so their size and difficulty are estimates rather than experience. **Contact Bas Spitters before choosing one**, so that we can agree the scope and adjust it if it turns out too large or too small.

You may choose these in place of the ones in the main list. They are of the same size: a few hundred lines of Rocq and one substantial idea, with the two type safety projects the largest. This document gives every definition you build on, and beyond them you need only the Rocq standard library. Each project ends with an exercise offering extensions, of which you complete at least one, and a harder extension earns a higher grade. Use the standard library wherever it helps, unless it trivialises the project: it holds sorted lists, finite maps and much else, and a development that instantiates a library module proves nothing of its own. Ask if you are unsure.

The exercises within a project are ordered: the early ones fix the definitions and should be quick, the middle ones are the ordinary work, and the last one or two before the extensions are where the project is won or lost. Work in that order.

Contents

1	Merge sort	3
2	Tries	4
3	Priority queues	5
4	Red-black trees	6
5	Abstract data types	7
6	Graph colouring	8
7	An optimiser for Imp	9
8	Verification conditions	10
9	Typing mutable references	11
10	Records and subtyping	12
11	A monadic interpreter for Imp	13
12	Ordered keys by type class	14

13 Information-flow control	15
14 Secure multi-execution	16
15 Constant-time programming	17
16 A reference monitor	18
17 Arbitrary-precision addition	19
18 Cipher modes	20
Acknowledgements	21

1 Merge sort

In this project you implement merge sort in Rocq and prove that it sorts.

A sorting proof has two halves: the output is ordered, and it has the same elements as the input. The second half is easy to forget, and without it a function returning the empty list would qualify.

Define `sorted` as an inductive proposition on `list nat`, and compare contents by counting:

```
Fixpoint count (x : nat) (l : list nat) : nat := (* fill out *).  
Definition same_contents (l1 l2 : list nat) : Prop :=  
  forall x, count x l1 = count x l2.
```

Exercise 1.1 Define `sorted` and `count`, and prove that `same_contents` is reflexive and transitive, and is preserved by adding the same element to both lists.

Exercise 1.2 Define `merge` on two lists. Rocq accepts a definition whose outer recursion is on the first list and whose inner recursion is on the second. Test it on examples, and prove `merge l [] = l`.

Exercise 1.3 Prove that merging preserves contents: `count x (merge l1 l2) = count x l1 + count x l2`. You will need a lemma unfolding `merge` on two non-empty lists: `simpl` exposes the inner `fixpoint` and makes the goal unusable.

Exercise 1.4 Prove that merging two sorted lists gives a sorted list.

Exercise 1.5 Define `split` and prove that it preserves contents and that both halves of a list of two or more elements are strictly shorter.

Exercise 1.6 Define merge sort with a step budget, in the style of `ceval_step`:

```
Fixpoint msort_aux (n : nat) (l : list nat) : list nat := (* fill out *).  
Definition msort (l : list nat) : list nat := msort_aux (length l) l.
```

Prove that with a budget at least the length of the list the result is sorted and has the same contents, and conclude the two theorems about `msort`.

Exercise 1.7 Extend the project in one of the following ways.

- Prove that sorted lists with the same contents are equal, and conclude that `msort` is idempotent (**moderate**).
- Generalise the development to any key type with a comparison operation, using a type class with the laws your proofs need (**moderate**).
- Count the comparisons the sort performs, by writing it in a writer monad, and prove a bound (**difficult**).
- Implement and verify a bottom-up merge sort, and prove it equivalent to this one (**difficult**).

2 Tries

In this project you implement a map indexed by bit strings and prove that it implements the same interface as the partial maps of the course.

Keys are lists of booleans, so the trie is a binary tree in which the path to a node spells out the key:

```
Definition key := list bool.
Inductive trie :=
| Leaf : trie
| Node : trie -> option nat -> trie -> trie.
```

Exercise 2.1 Define `lookup : key -> trie -> option nat` and `insert : key -> nat -> trie -> trie`, and test them.

Exercise 2.2 Prove `lookup k empty = None` and `lookup k (insert k x t) = Some x`.

Exercise 2.3 Prove that insertion leaves other keys alone:

```
Lemma lookup_insert_neq : forall k1 k2 x t,
  k1 <> k2 -> lookup k1 (insert k2 x t) = lookup k1 t.
```

This is the longest proof of the project. Case analysis on both keys and on the trie gives eight cases; do them separately rather than with a single tactic.

Exercise 2.4 Read the three lemmas together: define the map a trie denotes, and prove that insertion into the trie is update of the map. Conclude that the last insertion for a key wins.

Exercise 2.5 A trie can carry subtrees holding no values. Define `prune` removing them, and prove that pruning does not change any lookup, and that a trie with no values prunes to `Leaf`.

Exercise 2.6 Extend the project in one of the following ways.

- Add deletion, and prove the expected interaction with lookup (**moderate**).
- Prove that pruned tries are canonical: two pruned tries with the same lookups are equal (**difficult**).
- Allow any key type by a type class that encodes keys as bit strings, with instances for `nat` and for pairs (**moderate**).
- Write the lookup-then-update patterns with the option monad, and prove the monad laws for it (**moderate**).

3 Priority queues

In this project you specify a priority queue by the properties its operations must satisfy, implement it, and prove that the implementation meets the specification.

A priority queue supports `empty`, `insert` and `delete_max`. What matters is not how it is represented but that `delete_max` returns an element no smaller than any in the queue, and that the rest of the queue has lost exactly that one copy. Contents are compared by counting, as in the merge sort project.

Exercise 3.1 Define the queue as a list, with `insert` and a function `max_of : pq -> option nat`. Test them.

Exercise 3.2 Prove that `max_of q = None` exactly when the queue is empty, that the maximum is an element of the queue, and that it is an upper bound for the queue.

Exercise 3.3 Define `remove_one` removing a single occurrence of a value, and prove how it changes the count of each value.

Exercise 3.4 Define `delete_max` and prove the specification: the value returned is in the queue, it is maximal, and `count x q = count x q' + (if x =? m then 1 else 0)`.

Exercise 3.5 Drain the queue into a list by repeated `delete_max`, with a step budget as in merge sort, and prove the result is sorted in decreasing order. Prove it using the specification only, never the representation.

Exercise 3.6 Extend the project in one of the following ways.

- Give a second implementation — a sorted list, or a heap-ordered tree — and prove it meets the same specification (**moderate**).
- Prove that draining the queue gives a permutation of its contents, and hence that this is a sorting algorithm (**moderate**).
- Implement binomial queues and verify them (**very difficult**).
- Present the queue operations in a state monad, prove the monad laws, and restate the specification in terms of it (**difficult**).

4 Red-black trees

In this project you implement red-black tree insertion and prove that it keeps the tree ordered and balanced.

A red-black tree is a binary search tree whose nodes carry a colour, subject to two invariants: no red node has a red child, and every path from the root to a leaf passes the same number of black nodes. Insertion colours the new node red and repairs red-red violations on the way up with `balance`.

Exercise 4.1 Define `color`, `tree`, `member` and `balance`. The last has four interesting cases, one for each way a red child can have a red child.

Exercise 4.2 Define `ins` and `insert` (which paints the root black), and test insertion on examples.

Exercise 4.3 Define `ForallT` and the search tree property `BST` as inductive propositions, and prove that balancing preserves anything true of all keys.

Exercise 4.4 Prove that balancing preserves the search tree property, and then that insertion does. The case analysis is large; write a tactic that takes the hypotheses apart and rebuilds the result, and keep the arithmetic side conditions to `lia`.

Exercise 4.5 Define the colour invariants:

```
Inductive is_redblack : tree -> color -> nat -> Prop := (* fill out *).  
Inductive nearly_redblack : tree -> nat -> Prop := (* fill out *).
```

`is_redblack t c n` says `[t]` has black height `[n]` and, if its parent is red, a black root. `nearly_redblack` allows one red-red violation at the root, which is what `ins` may produce.

Exercise 4.6 Prove that balancing a black node whose child is nearly red-black restores the invariant, and conclude that insertion preserves it, with the black height rising by at most one.

Exercise 4.7 Extend the project in one of the following ways.

- Prove the height bound: a red-black tree of black height `n` has height at most `2n`, so operations are logarithmic (**difficult**).
- Implement deletion (**very difficult**).
- Generalise the key type with a type class carrying a lawful comparison (**moderate**).

5 Abstract data types

In this project you specify a finite set by equations, give two implementations, and prove that no client can tell them apart.

An implementation provides a carrier, an empty set, an insertion, a membership test, and a well-formedness predicate, and must satisfy `member x empty = false` and `member x (insert y s) = (x =? y) || member x s` for well-formed `s`. Package that as a record.

Exercise 5.1 *Define the record of operations and laws.*

Exercise 5.2 *Give an implementation by unordered lists and prove it meets the specification.*

Exercise 5.3 *Give an implementation by binary search trees and prove it meets the specification. The search tree property is needed to state well-formedness, although the membership equation turns out not to need it.*

Exercise 5.4 *Define what a client can do — build a set from a list of insertions and ask one membership question — and prove that what it observes is determined by the specification alone.*

Exercise 5.5 *Conclude that any two implementations agree on every client program, and instantiate the result at your two implementations.*

Exercise 5.6 *Extend the project in one of the following ways.*

- Add a third implementation by sorted lists (**moderate**); its membership proof is a long case analysis.
- Extend the interface with deletion and union, and redo the equivalence (**moderate**).
- Turn the record into a type class and let instance resolution choose the implementation; show that client code is unchanged (**moderate**).
- Prove that no client can distinguish the implementations even when the interface is extended with a fold (**difficult**).

6 Graph colouring

In this project you verify a graph colouring checker and a greedy colouring algorithm.

A graph is a list of edges over numbered vertices, and a colouring assigns a number to each vertex. A colouring is proper when no edge joins two vertices of the same colour.

Exercise 6.1 Define *proper* as a proposition and *check* as a boolean function, and prove the checker sound and complete with respect to it.

Exercise 6.2 Define the least colour not used by a given list of colours:

Definition `first_free (used : list nat) : nat := (* fill out *)`.

Prove that it is not in *used* and that it is at most `length used`. That a free colour exists at all is a counting argument: `length used + 1` candidates cannot all be used.

Exercise 6.3 Define the neighbours of a vertex and prove that both endpoints of an edge are neighbours of each other.

Exercise 6.4 Define greedy colouring: colour the vertices in increasing order, each with the least colour not used by its already-coloured neighbours. Prove that colours already assigned do not change as the colouring grows.

Exercise 6.5 Prove that the greedy colouring is proper. Colouring in a fixed order is what makes this short: for an edge, consider the endpoint coloured later.

Exercise 6.6 Prove the classical bound: greedy uses at most one more colour than the largest degree in the graph.

Exercise 6.7 Extend the project in one of the following ways.

- Prove that a graph with no edges needs one colour and that a complete graph on n vertices needs n (**moderate**).
- Verify a two-colouring algorithm for bipartite graphs, and prove that it fails exactly when the graph has an odd cycle (**very difficult**).
- Allow any vertex type with decidable equality, using a type class (**moderate**).

7 An optimiser for Imp

In this project you write an optimiser for Imp and prove that it does not change what programs do.

Two programs are equivalent when they relate the same states:

```
Definition cequiv (c1 c2 : com) : Prop :=  
  forall st st', ceval c1 st st' <-> ceval c2 st st'.
```

You will need the Imp of the course: `states`, `aexp`, `bexp`, `com` and the evaluation relation `ceval`. Restate them at the top of your file.

Exercise 7.1 *Prove the congruence lemmas: replacing a part of a command by an equivalent one gives an equivalent command, for assignment, sequencing and conditionals.*

Exercise 7.2 *Prove the congruence lemma for loops. This one does not follow the pattern of the others: the derivation has to be taken apart by induction with the loop **remembered**, and the proof is needed in both directions.*

Exercise 7.3 *Define constant folding on arithmetic expressions and prove it sound. Then do boolean expressions, where a folded comparison of two constants becomes **BTrue** or **BFalse**, and conjunction simplifies when one side is constant.*

Exercise 7.4 *Define constant folding on commands: a conditional whose guard folds to a constant loses a branch, and a loop whose guard folds to false disappears. Prove it sound. Use the congruence lemmas, and handle the two constant cases separately.*

Exercise 7.5 *Define a second pass removing **skip** from sequences, prove it sound, and compose the two passes into **optimise** with its soundness theorem. Give a program on which both passes do something.*

Exercise 7.6 *Extend the project in one of the following ways.*

- Add an optimisation of your own — unrolling a loop once, or removing an assignment whose variable is never read afterwards (**difficult**); the second needs a notion of liveness.
- Prove the optimiser idempotent (**moderate**).
- Record which optimisations fired, by writing the passes in a writer monad, and prove the record faithful (**difficult**).
- Prove that your optimiser never makes a program slower, for a suitable cost semantics (**very difficult**).

8 Verification conditions

In this project you build a verification condition generator for Imp and prove it sound, so that verifying a program becomes arithmetic.

An assertion is a predicate on states, and a Hoare triple is a statement about the evaluation relation:

Definition `Assertion := state -> Prop.`

Definition `hoare_triple (P : Assertion) (c : com) (Q : Assertion) : Prop :=
forall st st', ceval c st st' -> P st -> Q st'.`

The course does not cover Hoare logic, so this project includes reading about it; the Hoare chapters of *Programming Language Foundations* are the natural source.

Exercise 8.1 *Prove the rules for `skip`, assignment (in the backwards form), sequencing, conditionals and consequence, as theorems about `hoare_triple`.*

Exercise 8.2 *Prove the rule for loops: if the body preserves the invariant while the guard holds, the loop establishes the invariant with the guard false.*

Exercise 8.3 *Define annotated commands, in which each loop carries its invariant, and the function erasing the annotations.*

Exercise 8.4 *Define the precondition the generator computes, and the side conditions it produces. At a loop the precondition is the stated invariant, and the side conditions say that the body preserves it and that leaving the loop establishes the postcondition.*

Exercise 8.5 *Prove soundness: discharging the side conditions establishes the triple.*

Exercise 8.6 *Verify a program with it. A loop summing the numbers below n is enough; the side conditions should fall to `via`.*

Exercise 8.7 *Extend the project in one of the following ways.*

- Add arrays or a second variable discipline and extend the generator (**moderate**).
- Prove the generator complete for loop-free programs: if the triple holds, the side conditions do (**difficult**).
- Collect the side conditions in a writer monad and return them as a list rather than a conjunction (**moderate**).
- Prove a decorated-program checker correct, so that a program annotated at every step is verified by a single pass (**difficult**).

9 Typing mutable references

In this project you add a store to the simply typed lambda calculus and prove type safety again.

The course proves progress and preservation for the simply typed lambda calculus. Adding mutable references breaks the proof in an instructive way: the type of a term now depends on the store, so the invariant has to mention a store typing, and preservation has to say that the store keeps matching it. Keep the calculus small: unit, functions and references are enough.

Exercise 9.1 Define the syntax: types *Unit*, *Arrow* and *Ref*, and terms including *tm_ref*, *tm_deref*, *tm_assign* and locations *tm_loc*. Define substitution and values.

Exercise 9.2 Define stores as lists of terms, with lookup and update, and the step relation on pairs of a term and a store.

Exercise 9.3 Define store typings, the typing judgment (in which a location's type is read off the store typing), and what it means for a store to match a store typing.

Exercise 9.4 Prove the structural lemmas: typing is preserved by weakening the context, and substituting a closed well-typed value for a variable preserves typing.

Exercise 9.5 Prove that a term keeps its type when the store typing grows, and that allocation and assignment keep the store matching its typing.

Exercise 9.6 Prove preservation. State it over configurations — a pair of a term and a store — rather than over a term and a store separately, or the induction will not go through. The statement must say that the store typing grows:

```
exists ST', extends ST' ST /\ has_type ST' empty t' T /\  
store_well_typed ST' st'.
```

Exercise 9.7 Prove progress: a well-typed term is a value or steps, in any store matching the store typing.

Exercise 9.8 Extend the project in one of the following ways.

- Add booleans and conditionals, or natural numbers (**easy**).
- Add deallocation and show what goes wrong: give the typing rule and the configuration that breaks preservation (**moderate**).
- Add a typed *while* loop and prove that well-typed programs can diverge, so normalisation fails once references are present (**difficult**).
- Present the store discipline as a state monad and relate the two semantics (**very difficult**).

10 Records and subtyping

In this project you extend the simply typed lambda calculus with records and a subtyping relation, and prove type safety.

Subtyping lets a record with more fields stand in for one with fewer. Give the relation in syntax-directed form — no transitivity rule — so that inverting a subtyping derivation is immediate; transitivity is then a theorem.

Exercise 10.1 *Define types (unit, functions, and records as lists of labelled types) and terms (including record construction and projection), with substitution and values. A record is a value when all its fields are.*

Exercise 10.2 *Define the step relation, including the rule projecting a field from a record value and the rules stepping inside a record.*

Exercise 10.3 *Define subtyping: contravariant in argument types, covariant in result types, and on records comparing fields position by position, with the supertype allowed to stop early. Prove reflexivity. The induction principle Rocq generates for types does not reach inside the field list, so induct on the size of the type.*

Exercise 10.4 *Prove transitivity. Induction on either derivation fails in the function case, because the argument types go the other way; induct on the size of the middle type instead.*

Exercise 10.5 *Define typing with a subsumption rule, and prove the inversion lemmas: what a derivation for an abstraction, an application, a record or a projection tells you, given that subsumption may have been used last.*

Exercise 10.6 *Prove the canonical forms lemmas and then progress.*

Exercise 10.7 *Prove weakening and substitution, and then preservation and type safety.*

Exercise 10.8 *Extend the project in one of the following ways.*

- *Add permutation subtyping, so that field order does not matter (**very difficult**); you will need to rule out repeated labels.*
- *Add a top type and prove it is a supertype of everything (**easy**).*
- *Add variants (labelled sums) with their subtyping rule (**difficult**).*
- *Implement a subtyping checker and prove it sound and complete against the relation (**moderate**).*

11 A monadic interpreter for Imp

In this project you package the plumbing of an interpreter as a monad, prove the monad laws, and prove the interpreter correct.

The step-indexed evaluator of *ImpCEvalFun* threads a state and an **option** by hand. A monad hides that:

```
Definition M (A : Type) : Type := state -> option (A * state).
```

```
Definition ret {A : Type} (x : A) : M A := fun st => Some (x, st).
```

```
Definition bind {A B : Type} (m : M A) (f : A -> M B) : M B := (* fill out *).
```

Exercise 11.1 Define *ret*, *bind*, *fail*, *get* and *put*, with a notation for *bind*.

Exercise 11.2 Prove the three monad laws. They are equations between functions, so you need functional extensionality from the *Maps* chapter. Prove also that failure absorbs.

Exercise 11.3 Write the interpreter in the monad, with a step budget, and run it on a program.

Exercise 11.4 Prove it sound: if the interpreter returns a state, the evaluation relation relates the two states.

Exercise 11.5 Prove that a larger budget never turns success into failure.

Exercise 11.6 Prove completeness — every evaluation is found with some budget — and conclude that the interpreter and the relation agree.

Exercise 11.7 Extend the project in one of the following ways.

- Add a writer to the monad and make the interpreter log the assignments it performs; prove the log faithful (**moderate**).
- Add nondeterminism by moving to a list-valued monad, and extend *Imp* with a nondeterministic choice (**difficult**).
- Define a type class of monads with the laws as fields, instantiate it with this monad, *option* and the identity monad, and show which proofs can be done once for all instances (**moderate**).
- Prove that the interpreter is deterministic and derive determinism of the evaluation relation from it (**moderate**).

12 Ordered keys by type class

In this project you make the order on keys a parameter, as a type class with laws, and write a data structure and its proof once for all key types.

The search tree developments of the course fix `nat` as the key type and discharge the ordering side conditions with `lia`. A type class replaces both:

```
Class Ord (A : Type) := {
  cmp : A -> A -> comparison;
  cmp_refl : forall x, cmp x x = Eq;
  cmp_eq : forall x y, cmp x y = Eq -> x = y;
  cmp_antisym : forall x y, cmp x y = CompOpp (cmp y x);
  cmp_trans_lt : forall x y z, cmp x y = Lt -> cmp y z = Lt -> cmp x z = Lt }.
```

Exercise 12.1 Define the class and prove two consequences you will use: that equal elements compare `Eq`, and that `Lt` one way is `Gt` the other.

Exercise 12.2 Give the instance for `nat`, using the comparison lemmas of the standard library.

Exercise 12.3 Give the instance for pairs, ordered lexicographically, assuming instances for the components. The transitivity law is the one that takes work: case on the comparison of the first components.

Exercise 12.4 Define a search tree over an arbitrary ordered type, with membership and insertion.

Exercise 12.5 Prove the correctness of insertion using the class laws only — no arithmetic, no `lia`. Then use the result at both instances, and check that it computes on examples.

Exercise 12.6 Extend the project in one of the following ways.

- Add the search tree property and prove insertion preserves it, again generically (**moderate**).
- Add instances for lists (lexicographic) and for `option` (**moderate**).
- Derive a sorting function for any ordered type and prove it correct (**moderate**).
- Add a second class for monoids or monads, with laws, and show a function that is proved once and used at several instances (**difficult**).

13 Information-flow control

In this project you give *Imp* a security type system and prove that well-typed programs do not leak secrets.

Label every variable **Low** (public) or **High** (secret). An attacker sees only the public variables, so two states that agree on them look the same:

```
Definition low_eq (G : context) (st1 st2 : state) : Prop :=  
  forall x, G x = Low -> st1 x = st2 x.
```

Noninterference says that two runs of a program from low-equivalent states end low-equivalent: the attacker learns nothing about the secrets.

Exercise 13.1 Define labels, their join, and when one label may flow into another. Define the label of an arithmetic and of a boolean expression.

Exercise 13.2 Prove that a public expression evaluates the same in low-equivalent states.

Exercise 13.3 Define the type system. Assignment requires the expression's label, joined with the label of the context, to flow into the target; a conditional or loop raises the context label by the label of its guard. Give one program it rejects and one it accepts, and prove both claims.

Exercise 13.4 Prove confinement: a command typed in a **High** context leaves the public variables unchanged. This is what rules out leaks through the control flow.

Exercise 13.5 Prove noninterference. Induct on one evaluation and invert the other. The interesting cases are those where the two runs take different branches: there the guard must be secret, and confinement applies to both sides.

Exercise 13.6 Extend the project in one of the following ways.

- Add input and output, and prove noninterference for the output stream rather than the final state (**difficult**).
- Show that the type system is not complete: give a program that never leaks but is rejected (**easy**), and discuss why no sound type system can accept all of them.
- Replace the two-point lattice by a type class of lattices with the join and flow laws, instantiate it with a product lattice, and check that the proofs go through unchanged (**moderate**).
- Add declassification: a construct that deliberately leaks, with a policy saying what may be declassified, and prove the corresponding weaker property (**very difficult**).

14 Secure multi-execution

In this project you enforce noninterference without a type system, by running every program twice.

Instead of rejecting programs that might leak, run the program once on a state whose secrets have been replaced by defaults — that run decides the public outputs — and once on the real state, which decides the secret outputs. The public half then cannot depend on the secrets, whatever the program does.

Exercise 14.1 *Define labels, low equivalence, and the projection replacing every secret by a default. Prove that low-equivalent states have equal projections. States are functions, so use functional extensionality from the Maps chapter.*

Exercise 14.2 *Prove that `Imp` is deterministic.*

Exercise 14.3 *Define secure multi-execution as a relation on an initial and a final state, and run it on a program that leaks under the ordinary semantics. Prove that the public variable comes out the same whatever the secret was.*

Exercise 14.4 *Prove the theorem: for every program, multi-execution is noninterfering.*

Exercise 14.5 *Prove transparency: when every variable is public, multi-execution agrees with the ordinary semantics. A mechanism that changed the meaning of innocent programs would be useless.*

Exercise 14.6 *Extend the project in one of the following ways.*

- *Prove transparency for programs that are already noninterfering, not only for those with no secrets (**very difficult**).*
- *Extend to more than two levels, with one run per level (**difficult**).*
- *Compare with the type system of the information-flow project: give a program accepted by one and rejected by the other, in both directions (**moderate**).*
- *Present the paired runs as a reader monad over the security level (**moderate**).*

15 Constant-time programming

In this project you formalise a timing side channel and enforce its absence with a type system.

An attacker who cannot read memory may still see which way each branch went, by timing the program or by watching the instruction cache. Model that with a leakage trace: instrument the evaluation relation so that it also produces the list of guard outcomes. A program is constant-time when two runs from states agreeing on the public variables leak the same trace.

Exercise 15.1 *Define the instrumented relation $\text{ceval } c \text{ st } st' \ t$, recording a boolean for every conditional and every loop test.*

Exercise 15.2 *Define labels and low equivalence, and prove that public expressions evaluate equally in low-equivalent states.*

Exercise 15.3 *Define the type system: no branching on a secret, and nothing secret written to a public variable. Show a program it rejects — the shape of a timing leak — and a program it accepts that still computes with secrets.*

Exercise 15.4 *Prove the security theorem: two runs of a well-typed program from low-equivalent states leak the same trace and end low-equivalent. Both halves are needed at once: the states must stay equivalent for the later guards to agree.*

Exercise 15.5 *State constant-time security as a property of the program alone, and derive it from the theorem.*

Exercise 15.6 *Extend the project in one of the following ways.*

- Add arrays and make the trace record the indices read and written, which is the cache side channel; extend the type system and the proof (**difficult**).
- Add a constant-time conditional (a `select` instruction) and show it accepts programs the branch-based one rejects (**moderate**).
- Formalise speculative execution — a semantics that may take the wrong branch and roll back — and show the type system no longer suffices (**very difficult**).
- Present the instrumented semantics as a writer monad over traces (**moderate**).

16 A reference monitor

In this project you verify a monitor that stops a program from doing what a policy forbids, and show that it leaves everything else alone.

A monitor is only half useful if it is only sound: a monitor that blocks every program is sound. The other half is transparency — programs that never violate the policy must run exactly as they would without it.

Exercise 16.1 *Define a policy saying which variables may be written, and the monitored evaluation relation: the ordinary one, with a side condition on assignment.*

Exercise 16.2 *Give a program that writes a protected variable, and prove that the monitor has no derivation for it.*

Exercise 16.3 *Prove soundness: a monitored run never changes a protected variable.*

Exercise 16.4 *Define what it means for a program to comply with the policy, and prove transparency: a complying program has the same monitored runs as ordinary runs, in both directions.*

Exercise 16.5 *Define a second monitor that runs a handler instead of getting stuck, and prove that it is still sound — the handler is monitored too, so it cannot write protected variables either.*

Exercise 16.6 *Extend the project in one of the following ways.*

- *Make the policy depend on the state, so that a variable becomes writable only after a check, and prove soundness again (**moderate**).*
- *Add a history: the policy constrains sequences of operations rather than single ones, and the monitor carries the history (**difficult**).*
- *Prove that no monitor can enforce a policy that constrains what a program would have done in another run, which is why noninterference needs a different technique (**very difficult**).*
- *Present the blocking monitor as an exception monad (**moderate**).*

17 Arbitrary-precision addition

In this project you verify an addition on numbers represented as digit lists, in the style of the *bignum* code in cryptographic libraries.

A number is a little-endian list of digits in a fixed base, and the function to verify is addition with carry propagation. You prove two things: the result denotes the sum, and every digit of the result is in range.

Exercise 17.1 Fix a base, define the value a digit list denotes, and define well-formedness: every digit is below the base. Test the value function.

Exercise 17.2 Define carry propagation into a single number, and addition of two numbers with an incoming carry. Test them, including a case where the result is longer than both inputs.

Exercise 17.3 Prove the division identity you will need, and that the remainder is below the base. Be warned that *simpl* unfolds *Nat.modulo* into its implementation and makes the goal unusable: state one-step unfolding lemmas for your own functions and use those instead.

Exercise 17.4 Prove that carry propagation adds the carry to the value, and that addition computes the sum of the two values plus the incoming carry.

Exercise 17.5 Prove that the result is well formed. The invariant on the carry is that it is at most one, not merely that it is below the base: two digits and a carry come to at most twice the base minus one.

Exercise 17.6 Extend the project in one of the following ways.

- Add subtraction, with the precondition that the result is non-negative, and prove it correct (*moderate*).
- Add schoolbook multiplication and prove it correct (*difficult*).
- Add comparison and prove it agrees with the comparison of the values (*moderate*).
- Make the base a parameter with a type class carrying the laws your proofs use, and instantiate it at two bases (*moderate*).
- Prove that normalised representations are unique, so equality of values is equality of digit lists (*difficult*).

18 Cipher modes

In this project you verify the plumbing around a block cipher: that a mode of operation inverts, that padding is unambiguous, and that the simplest mode leaks.

The block cipher itself is a parameter. Assume only that decryption inverts encryption on a single block, and prove everything else. Blocks are natural numbers and exclusive or is `Nat.lxor`, so no bit-vector bookkeeping is needed. This project verifies the *construction*, not the cipher: nothing here says anything about secrecy, and the report should say so.

Exercise 18.1 *Prove that exclusive or cancels: $\text{lxor} (\text{lxor } a \ b) \ b = a$.*

Exercise 18.2 *Define electronic codebook mode and prove it inverts. Then prove the leak: equal plaintext blocks always give equal ciphertext blocks, so the ciphertext reveals which blocks of the plaintext are equal.*

Exercise 18.3 *Define cipher block chaining, with each ciphertext block fed into the next block's encryption, and its decryption.*

Exercise 18.4 *Prove that chaining inverts, for any initialisation vector.*

Exercise 18.5 *Define padding that extends a message to a multiple of the block size by appending n copies of n , and the unpadding that removes it. Prove the round-trip, and conclude that padding is injective, so a padded message determines the message.*

Exercise 18.6 *Extend the project in one of the following ways.*

- *Verify counter mode, and prove that it inverts without assuming the cipher is invertible (**moderate**).*
- *Prove that unpadding rejects malformed padding, and discuss why a decryption that reports the difference is a padding oracle (**difficult**).*
- *Verify a message authentication code built from the block cipher, and prove that verification accepts exactly the tags the code produces (**moderate**).*
- *Thread the chaining value through a state monad, define the modes as a traversal in it, and prove the result equals the explicit recursion (**moderate**).*

Acknowledgements

The data structure projects follow the material of *Verified Functional Algorithms*, and the language projects that of *Programming Language Foundations*, both volumes of *Software Foundations* by Benjamin C. Pierce and others. The security projects follow *Security Foundations*, the Rocq volume by Cătălin Hrițcu and Yonghyun Kim.